



Rapport de stage

Développeur Python

Vanlaer Louis

De juin 2026 à août 2026

Tuteur de stage : Taya MAHENDRARAJAH

Établissement : EPITA La Défense – Bachelor 2 (2025-2026)

Entreprise d'accueil : LSEG – Refinitiv, équipe ACE, Paris (France)

Remerciements

Avant toute chose, je souhaite remercier l'ensemble des personnes qui m'ont accompagné, de près ou de loin, dans ma recherche de stage et qui m'ont permis de travailler dans les meilleures conditions.

J'adresse tout particulièrement mes sincères remerciements à Taya Mahendrarajah, Aissata Diop, Filippo Kassab, Stéphane Tinseau, François Thou, ainsi qu'à l'ensemble des membres de l'équipe ACE de LSEG.

Je remercie tout particulièrement Mme Mahendrarajah pour sa bienveillance, sa confiance et la qualité de son accompagnement tout au long de mon stage. Grâce à ses conseils, j'ai pu approfondir mes compétences en Python et affiner mon projet professionnel. Pour une première expérience professionnelle dans le domaine de l'informatique, je ne pouvais espérer un meilleur encadrement.

Je remercie également M. Kassab et Mme Diop de m'avoir accueilli au sein de l'équipe et de m'avoir donné l'opportunité de réaliser ce stage.

Je tiens aussi à remercier M. Tinseau pour les connaissances qu'il m'a transmises en Linux et en Python, ainsi que M. Aurélien Capseck pour ses conseils concernant mes choix d'orientation.

Je souhaite également remercier les équipes administratives de LSEG, qui ont facilité mes démarches et m'ont apporté leur aide chaque fois que cela a été nécessaire.

Enfin, je remercie ma famille pour son soutien constant dans mes démarches de recherche de stage et dans mes choix d'orientation.

Les fonctions des principales personnes mentionnées dans ce rapport sont présentées dans l'organigramme de l'équipe ACE figurant dans les annexes.

Sommaire

Remerciements	2
Introduction	5
1 - Présentation de l'entreprise	5
1.1 - Présentation de LSEG	5
1.2 - Environnement de stage	6
2 - Les étapes du projet	6
2.1 - Importation et préparation des données (cf. annexe A)	6
2.2 - Configuration de l'application et bouton Run (cf. annexes B et C).....	6
2.3 - Création et gestion des tableaux de données (cf. annexes D et E)	6
2.4 - Développement des graphiques de visualisation par échéance (cf. annexes F, G et H) ...	7
2.5 - Contrôle qualité à l'aide d'indicateurs de Goodness of Fit (cf. annexes I, J et K).....	7
2.6 - Synthèse des contrats après filtrage (cf. annexe L)	8
2.7 - Export des résultats vers Excel (cf. annexes M et N).....	8
2.8 - Communication professionnelle et documentation (cf. annexe O).....	8
3 - Difficultés rencontrées et solutions apportées.....	9
3.1 - Restructuration du code pour l'intégration avec Streamlit.....	9
3.2 - Gestion du bouton « Run » et de l'état de l'application	9
3.3 - Adaptation à de nouvelles technologies et contraintes organisationnelles	9
Conclusion : bilan et perspectives	9
Bibliographie	10
Annexes	13
Organigramme de l'équipe ACE	13
Annexe A – Construction et exécution des requêtes de calcul à partir d'un modèle JSON.....	14
Annexe B – Configuration des inputs.....	14
Annexe C – Développement de l'interface utilisateur et gestion des paramètres de calcul.....	15
Annexe D – Construction et traitement de la table de comparaison	16
Annexe E – Tableau de comparaison flouté des instruments dépassant un seuil de tolérance	17
Annexe F – Comparaison graphique des volatilités de marché et du modèle SVI	17
Annexe G – Analyse de la répartition des contrats autour de l'ATM	18
Annexe H - Fonction de génération du graphique comparatif entre données de marché et courbes SVI.....	19
Annexe I – Tableau des résultats SVI par échéance.....	19
Annexe J – Analyse du Goodness of Fit.....	20
Annexe K – Résultats du Goodness of Fit par échéance	20
Annexe L – Informations de synthèse de la surface de volatilité.....	21

Annexe M – Exportation des résultats vers Excel	22
Annexe N – Rapport Excel consolidé contenant les résultats de comparaison et de calibration	23
Annexe O – Documentation en Markdown	24
Annexe P – Bibliothèques Python utilisées.....	24
Annexe Q – Espace de travail au sein des locaux de LSEG à Paris	25

Introduction

Du 29 juin 2026 au 28 août 2026, j'ai effectué un stage au sein de Refinitiv, entité du London Stock Exchange Group (LSEG). Cette expérience m'a permis d'évoluer dans un environnement spécialisé dans les données et les services financiers.

La mission qui m'a été confiée consistait à développer en Python une interface utilisateur destinée au contrôle qualité d'une surface de volatilité implicite.

L'objectif de cette application était de faciliter l'analyse, la visualisation et la validation de données de marché au moyen de différents outils de traitement et de contrôle.

Dans ce rapport, je présenterai tout d'abord le groupe LSEG ainsi que l'équipe au sein de laquelle j'ai été intégré.

Je détaillerai ensuite les étapes du développement de l'application :

- L'importation des données d'entrée (requêtes JSON, dates et instruments financiers)
- Le développement de l'interface de configuration de l'application et du bouton Run, permettant de déclencher les traitements et de générer les résultats
- La création et la gestion des tableaux de données (DataFrames, fusions de données, calculs de tolérances, etc.)
- Le développement de graphiques de visualisation, notamment les courbes de volatilités (*smiles*) avec sélection multiple ;
- L'implémentation d'indicateurs de contrôle qualité (*Goodness of Fit*)
- L'affichage d'informations de synthèse sur les contrats après filtrage
- L'exportation des résultats vers Excel
- La rédaction de la documentation technique du projet (README)

Dans une troisième partie, j'exposerai les principales difficultés rencontrées au cours du stage ainsi que les solutions mises en œuvre pour les surmonter.

Enfin, je présenterai les principaux enseignements tirés de cette expérience, tant sur le plan professionnel que personnel, ainsi que son influence sur mon projet de carrière.

1 - Présentation de l'entreprise

1.1 - Présentation de LSEG

LSEG (London Stock Exchange Group) est un groupe international spécialisé dans les infrastructures de marchés financiers, les données financières et les services d'analyse. Présent à l'échelle mondiale, il joue un rôle majeur dans le fonctionnement des marchés en permettant aux entreprises, aux investisseurs et aux institutions financières d'accéder aux marchés de capitaux, de gérer leurs risques et d'exploiter des données financières de référence.

Le siège social du groupe est situé à Londres, au Royaume-Uni.

Le groupe s'organise autour de plusieurs divisions majeures, parmi lesquelles Data & Analytics, FTSE Russell, Capital Markets, Post Trade et Risk Intelligence.

Ces différentes entités collaborent afin de fournir aux acteurs des marchés financiers des services de données, d'indices, de négociation, de compensation et de gestion des risques.

1.2 - Environnement de stage

J'ai réalisé mon stage au sein des bureaux parisiens de LSEG, situés au Centorial, 18 rue du Quatre-Septembre (75002 Paris). J'ai intégré l'équipe ACE (Analytics Center of Excellence) dans le pôle LSEG – Data & Analytics, dont l'activité est centrée sur la production et l'analyse de données financières. Une vue de l'espace de travail est présentée en annexe Q.

2 - Les étapes du projet

2.1 - Importation et préparation des données (cf. annexe A)

La première étape du projet a consisté à préparer les données nécessaires aux calculs à partir d'un modèle de requête JSON. Le fichier est d'abord chargé par la fonction `load_json_from_file()`, puis la fonction `get_request()` adapte les principaux paramètres : l'instrument financier, la date de trading et le Price Side, c'est-à-dire le type de prix utilisé pour la contribution des données (BID, ASK, MID ou LAST).

Une fois la requête configurée, la fonction `get_response2()` l'envoie au service de calcul et récupère la réponse JSON. Les données obtenues servent ensuite de base aux traitements, aux comparaisons et aux contrôles réalisés dans les étapes suivantes (cf. annexe A).

2.2 - Configuration de l'application et bouton Run (cf. annexes B et C)

La deuxième étape du projet a consisté à développer, avec Streamlit, une interface de configuration permettant à l'utilisateur de renseigner les paramètres nécessaires au calcul. Il peut charger le modèle de requête JSON, saisir l'instrument financier, définir le seuil de tolérance, sélectionner deux dates de trading et préciser le Price Side associé à chacun des deux scénarios (cf. annexes B et C).

Une fois ces paramètres renseignés, l'utilisateur lance les traitements à l'aide du bouton « Run ». Ce fonctionnement permet de séparer clairement la phase de configuration de la phase d'exécution et d'éviter de déclencher les calculs avant que l'ensemble des paramètres ne soit défini.

J'ai également ajouté plusieurs contrôles de saisie afin de limiter les erreurs et d'améliorer l'expérience utilisateur, notamment pour la validation du seuil de tolérance et le chargement du fichier JSON.

2.3 - Création et gestion des tableaux de données (cf. annexes D et E)

La troisième étape a consisté à construire une table permettant de comparer les données de volatilité obtenues pour les deux scénarios sélectionnés.

À partir des réponses JSON, les informations relatives aux options — instrument, prix de l'option, strike, échéance, volatilité et type — sont extraites puis converties en DataFrames Pandas par la fonction `read_data_createDataframe()` (cf. annexe D).

Les deux jeux de données sont ensuite fusionnés à l'aide de la fonction `merge_data()` afin d'identifier les contrats présents dans les deux scénarios ainsi que ceux n'apparaissant que dans l'un d'eux. Les données obtenues sont ensuite nettoyées et réorganisées avec la fonction `clean_merged_data()` afin de rendre le tableau final plus clair et plus lisible.

Pour les contrats communs aux deux scénarios, l'application calcule ensuite l'écart relatif de volatilité à l'aide des fonctions `difference()` et `tolerance()`. Cet écart est comparé au seuil de tolérance défini par l'utilisateur.

Lorsque le seuil est dépassé, la ligne correspondante est mise en évidence en rouge grâce à la fonction `highlight_tolerance()`. Le tableau obtenu permet ainsi d'identifier rapidement les écarts nécessitant une vérification particulière (cf. annexe E).

L'intérêt de cette vue est de concentrer l'attention sur les différences significatives plutôt que d'imposer une vérification manuelle de l'ensemble des contrats.

Elle constitue ainsi un premier niveau de contrôle qualité entre les deux scénarios comparés.

2.4 - Développement des graphiques de visualisation par échéance (cf. annexes F, G et H)

Pour chaque échéance sélectionnée, l'application génère un graphique permettant de comparer les données de marché des deux dates de trading (cf. annexe F).

La fonction `create_plot()` trie d'abord les données par strike, puis représente les volatilités de marché des deux scénarios sous forme de points. Les courbes issues du modèle SVI sont ensuite ajoutées lorsqu'elles sont disponibles pour l'échéance concernée. Le strike est placé sur l'axe des abscisses et la volatilité sur l'axe des ordonnées.

Deux lignes verticales indiquent le prix du sous-jacent pour chacun des scénarios. Elles permettent de situer rapidement la zone At-The-Money (ATM), correspondant aux strikes proches du prix du sous-jacent. Le graphique est ensuite complété par une légende, une grille et un titre avant d'être affiché dans l'application (cf. annexe H).

Cette représentation complète le tableau de comparaison en permettant d'identifier visuellement un déplacement de la courbe, une différence entre les deux dates ou encore un écart entre les données de marché et le modèle SVI.

Sous chaque graphique, un tableau récapitulatif présente la répartition des contrats selon leur distance par rapport à l'ATM. Les strikes sont répartis en trois catégories : plus de 20 % sous l'ATM, autour de l'ATM et plus de 20 % au-dessus de l'ATM. Cette vue permet de mieux comprendre la répartition des données utilisées pour l'analyse de chaque échéance (cf. annexe G).

2.5 - Contrôle qualité à l'aide d'indicateurs de Goodness of Fit (cf. annexes I, J et K)

Les résultats de calibration SVI sont d'abord présentés sous forme de tableaux Pandas pour chaque scénario. Un dégradé de couleurs facilite la lecture des valeurs et permet de repérer plus rapidement les différences entre échéances (cf. annexe I).

Afin de compléter cette analyse, un indicateur de Goodness of Fit a été intégré à l'application. La logique de classification utilisée pour interpréter cet indicateur est présentée en annexe J.

Cet indicateur s'appuie sur la métrique Average Spread Explained, calculée lors de la calibration de la surface de volatilité. Elle permet d'évaluer, pour chaque échéance, la qualité de l'ajustement entre les données observées sur le marché et la courbe obtenue avec le modèle SVI.

Dans l'application, les résultats sont présentés au moyen d'un code couleur allant du vert pour les ajustements les plus satisfaisants au rouge pour ceux nécessitant davantage d'attention (cf. annexe K).

2.6 - Synthèse des contrats après filtrage (cf. annexe L)

L'application affiche également des informations de synthèse sur la surface de volatilité après les différentes étapes de filtrage et de calibration.

Cette vue indique notamment le nombre initial de contrats disponibles (`initialCount`) ainsi que le nombre de contrats finalement retenus pour la construction de la surface (`finalCount`) (cf. annexe L).

La comparaison de ces deux valeurs permet d'estimer le nombre de contrats écartés au cours du traitement et de mieux comprendre l'effet des filtres appliqués par le service de calcul.

Pour des raisons de confidentialité, seule une partie de la capture d'écran est présentée et certaines informations ont volontairement été masquées.

2.7 - Export des résultats vers Excel (cf. annexes M et N)

Afin de faciliter l'exploitation des résultats en dehors de l'application, j'ai développé une fonction d'export vers Excel à l'aide de Pandas et d'ExcelWriter. La fonction `export_to_excel()` regroupe les différents DataFrames dans un même fichier (cf. annexe M).

Le classeur généré contient notamment les données de comparaison, les contrats dépassant le seuil de tolérance, les résultats de calibration SVI ainsi que les indicateurs de Goodness of Fit.

Les contrats présents dans un scénario mais absents de l'autre sont également recensés dans un onglet dédié (`df_nan`), afin de faciliter l'identification des différences entre les deux dates comparées.

Chaque jeu de données est exporté dans une feuille distincte afin de garantir une consultation claire et structurée des résultats. Un exemple du fichier Excel consolidé est présenté en annexe N.

2.8 - Communication professionnelle et documentation (cf. annexe O)

Afin de faciliter la reprise et la maintenance du projet, j'ai ajouté des commentaires dans le code pour en améliorer la lisibilité, puis rédigé une documentation technique sous la forme d'un fichier README.

Cette documentation présente les fonctionnalités de l'application, les principales étapes d'utilisation ainsi que les éléments nécessaires à son exécution (cf. annexe O).

Ce travail m'a permis de mieux comprendre l'importance de la transmission des connaissances et de la documentation dans un projet de développement.

3 - Difficultés rencontrées et solutions apportées

3.1 - Restructuration du code pour l'intégration avec Streamlit

Au début du projet, j'ai développé le script de manière procédurale, sans structurer le code en fonctions. Lors de l'intégration de l'interface utilisateur avec Streamlit, j'ai constaté qu'une organisation plus modulaire était nécessaire afin de mieux gérer l'exécution du programme et les interactions avec l'interface. J'ai donc restructuré une partie importante du code existant en fonctions réutilisables, ce qui a amélioré sa lisibilité, sa maintenance et son intégration dans Streamlit.

3.2 - Gestion du bouton « Run » et de l'état de l'application

J'ai également rencontré des difficultés lors de l'implémentation du bouton Run. À chaque modification d'un paramètre, comme l'échéance, la tolérance, les dates ou le côté de prix, l'application se réinitialisait et revenait à son état initial. Après plusieurs essais, j'ai résolu ce problème en utilisant `st.session_state`. Ce mécanisme m'a permis de conserver l'état de l'application et de déclencher les calculs uniquement après un clic sur le bouton Run.

3.3 - Adaptation à de nouvelles technologies et contraintes organisationnelles

Mon tuteur de stage étant parfois mobilisé par des périodes de forte activité, le temps disponible pour l'accompagnement pouvait être limité. J'ai donc dû gagner rapidement en autonomie, prendre des notes précises et m'adapter aux nouvelles demandes qui m'étaient transmises. Cette situation m'a également conduit à découvrir et à utiliser de nouveaux outils et méthodologies, notamment Streamlit, que je ne connaissais pas auparavant.

Conclusion : bilan et perspectives

Au cours de ce stage, j'ai eu l'opportunité de concevoir et de développer une interface utilisateur en Python destinée à assister un analyste dans le contrôle et l'analyse de données de marché. Bien que Python ait déjà été abordé durant ma première année de formation, ce projet m'a permis d'approfondir mes connaissances et de les mettre en pratique dans un contexte professionnel.

J'ai notamment travaillé sur l'intégration de données, la création de tableaux de comparaison, la visualisation graphique des résultats, l'implémentation d'indicateurs de contrôle qualité ainsi que l'export des données vers Excel.

Cette expérience a également renforcé mon autonomie et ma capacité à résoudre des problèmes. Comme dans tout projet de développement, la réalisation de l'application a été marquée par plusieurs difficultés techniques. J'ai appris à utiliser des outils de débogage, à rechercher des solutions adaptées et à persévérer face aux obstacles rencontrés. Ces situations

m'ont permis de développer davantage ma rigueur, ma méthode de travail et mon esprit d'analyse.

Sur le plan humain, ce stage m'a permis de mieux comprendre les relations professionnelles au sein d'une équipe ainsi que l'importance de la communication, de l'entraide et du respect mutuel dans un environnement de travail. L'ambiance à la fois sérieuse et conviviale a facilité mon intégration et m'a permis de gagner en confiance tout au long de cette expérience.

J'ai particulièrement apprécié le fait de travailler avec des données financières et de mieux comprendre leur utilisation dans un environnement professionnel.

Le développement logiciel et la programmation m'ont intéressé. Mais à terme, j'aimerais idéalement évoluer dans un domaine situé à la croisée de la donnée et de la cybersécurité et peut être de la finance, plutôt que du pur développement. Les prochains mois me permettront donc d'approfondir cette orientation et d'identifier les métiers correspondant le mieux à mes compétences et à mes aspirations.

En conclusion, ce stage chez LSEG – Refinitiv constitue une étape importante de mon parcours. Il m'a permis d'acquérir une première expérience concrète du monde professionnel, de développer de nouvelles compétences techniques et humaines et de mieux définir mes objectifs pour la suite de mes études et de ma carrière.

À l'issue du stage, le projet aboutit à une application structurée permettant de configurer deux scénarios, comparer les données obtenues, visualiser les écarts et les courbes de volatilité, contrôler la qualité de calibration, identifier certains contrats absents ou filtrés et exporter les résultats vers Excel.

La prochaine étape consisterait idéalement à permettre la comparaison des données sur une plage de dates plus étendue, tout en conservant un temps de calcul suffisamment court pour garantir une utilisation fluide de l'application. Cette évolution pourrait être poursuivie par une personne disposant de davantage d'expérience technique afin d'exploiter pleinement le potentiel du projet.

La documentation que j'ai rédigée devrait faciliter cette reprise en présentant le fonctionnement de l'application, son organisation et les principales étapes nécessaires pour poursuivre le développement du code.

À ce stade, l'application peut être utilisée localement par certains membres de l'équipe pour des besoins ponctuels. À terme, l'objectif serait de versionner et de déployer le code sur un serveur interne via Git afin de rendre l'outil plus facilement accessible. Cette mise en production nécessite toutefois des droits d'accès dont je ne dispose pas et devra donc être réalisée par mon tuteur.

Bibliographie

Volatilité et modèle SVI

Gatheral, J. (2004). *A Parsimonious Arbitrage-Free Implied Volatility Parameterization*.
Merrill Lynch Quantitative Research.

Référence fondamentale sur le modèle SVI utilisé pour représenter les surfaces de volatilité.

CBOE. VIX Index and Volatility Resources. Disponible sur :

<https://www.cboe.com>

Source reconnue pour les notions de volatilité implicite et les indicateurs de marché liés à la volatilité.

Développement de l'application

Streamlit Inc. Streamlit Documentation. Disponible sur :

<https://docs.streamlit.io>

Documentation officielle utilisée pour le développement de l'interface utilisateur.

The Pandas Development Team. *Pandas Documentation*. Disponible sur :

<https://pandas.pydata.org/docs/>

Documentation utilisée pour la manipulation et l'analyse des données.

Matplotlib Development Team. Matplotlib Documentation. Disponible sur :

<https://matplotlib.org/stable/>

Documentation utilisée pour la création des graphiques de visualisation.

Webographie

Microsoft Learn. Disponible sur :

<https://learn.microsoft.com>

Documentation technique consultée pour certaines fonctionnalités Python et développement.

Data Professor. *Build Data Apps in Python with Streamlit* [Vidéo YouTube].

Tutoriel utilisé lors de la prise en main de Streamlit.

Méthodologie de rapport de stage

Université Paris Cité. *Guide de rédaction du rapport de stage*. Disponible sur :

https://u-paris.fr/eila/wp-content/uploads/sites/42/2021/07/Consignes_Rapport_de_Stage.pdf

Guide méthodologique consulté pour la rédaction du rapport de stage.

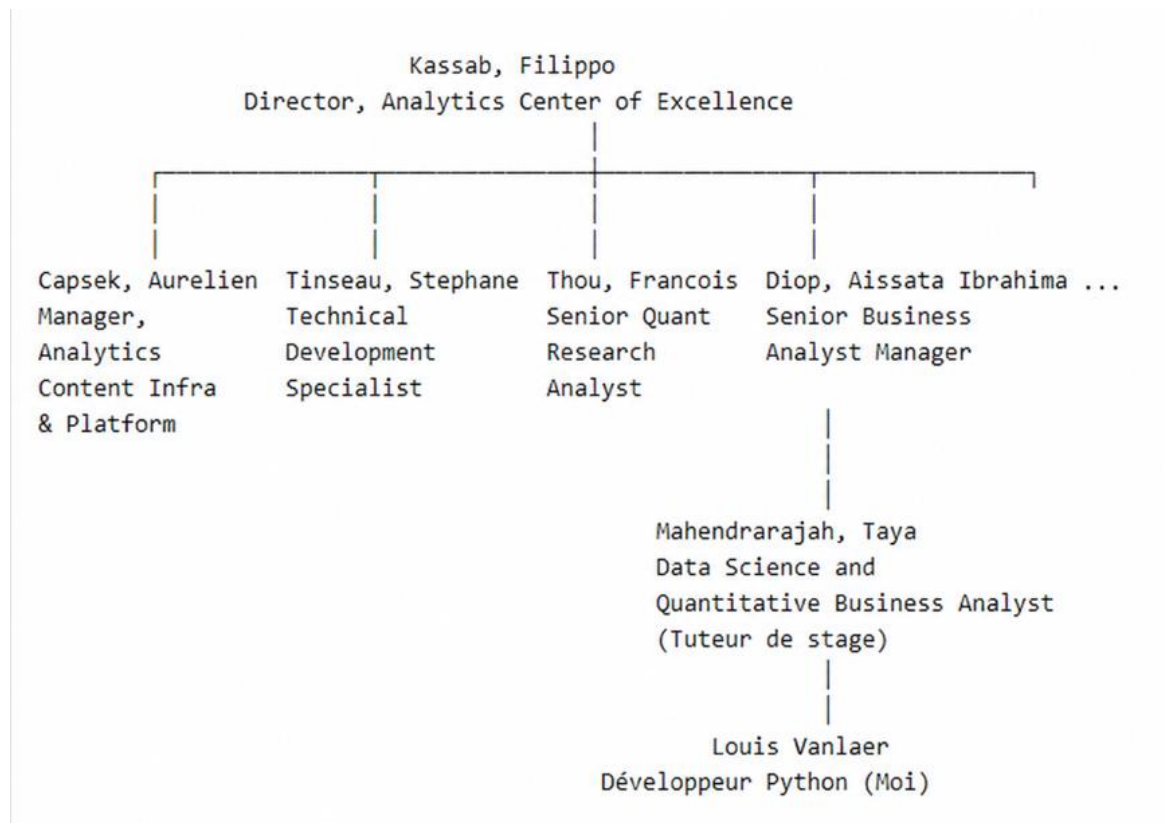
Compilatio.net. Comment faire un rapport de stage étape par étape? Disponible sur :

<https://www.compilatio.net/blog/comment-rediger-un-rapport-de-stage>

Ressource méthodologique consultée pour structurer et rédiger un rapport de stage.

Annexes

Organigramme de l'équipe ACE



Annexe A – Construction et exécution des requêtes de calcul à partir d'un modèle JSON

```
def load_json_from_file(file_path: str) -> dict:
    try:
        with open(file_path, 'r') as f:
            return json.load(f)

    except (OSError, TypeError) as e:
        raise FileNotFoundError(
            f"Impossible de charger le fichier : {file_path}"
        ) from e

    except json.JSONDecodeError as e:
        raise ValueError(
            f"JSON invalide : {file_path}"
        ) from e

def get_request(path, trade_date, instrument, PriceSide):
    basic_request = load_json_from_file(path)

    print("path =", path)
    print("basic_request =", basic_request)

    request = copy.deepcopy(basic_request)

    request['universe'][0]['surfaceParameters']['calculationDate']=trade_date
    request['universe'][0]['surfaceParameters']['priceSide']=PriceSide
    request['universe'][0]['surfaceParameters']['debugMode']="True"
    request['universe'][0]['underlyingDefinition']['instrumentCode']=instrument+"@RIC"
    request['outputs'][0]="GoodnessOfFit"
    request['outputs'][0]="SurfaceInformation"

    return request

def get_response2(requestQPS):
    qps_request = copy.deepcopy(requestQPS)
    json_response = json.loads(requests.post(url, data=json.dumps(qps_request), headers=Headers_debug).text)
    return json_response
```

Annexe B – Configuration des inputs

Configuration

Upload json request

stocks.json

1.4KB

+

Enter the instrument:

IBM

Tolerance(%)

1

CASE 1

CASE 2

Date 1

Date 2

2026/08/24

2026/08/25

PriceSide 1

PriceSide 2

Mid

Mid

Run

Annexe C – Développement de l'interface utilisateur et gestion des paramètres de calcul

```
#####Config#####
with st.sidebar:
    st.set_page_config(layout="wide")
    st.header("Configuration")
    st.subheader("📁 Upload json request")

    try:
        uploaded_file = st.file_uploader("", type=["json"], on_change=_reset_run)
        request_path = None

        if uploaded_file is not None:

            root_dir = Path.cwd()

            request_path = root_dir / uploaded_file.name

            with open(request_path, "wb") as f:
                f.write(uploaded_file.getbuffer())

    except AttributeError:
        print("choose json file")
    ##### choose Instrument

    choose_instrument = st.text_input("Enter the instrument:", on_change=_reset_run)
    ##### Tolerance Inputs
    threshold = 0
    tolerance_selected = st.text_input("Tolerance(%)", on_change=_reset_run)
    if tolerance_selected:
        try:
            tolerance_value = float(tolerance_selected)
            threshold = tolerance_value
        except ValueError:
            st.error("Enter a valid number")

    col1, col2 = st.columns(2)

    with col1:
        st.header("CASE 1")
        date_1 = st.date_input("Date 1", value=date.today(), max_value=date.today(), on_change=_reset_run)
        # Weekends are not allowed
        priceSide1 = st.text_input("PriceSide 1", on_change=_reset_run)

    with col2:
        st.header("CASE 2")
        date_2 = st.date_input("Date 2", value=date.today(), max_value=date.today(), on_change=_reset_run)
        # Weekends are not allowed
        priceSide2 = st.text_input("PriceSide 2", on_change=_reset_run)

    if st.button("🏃 Run", use_container_width=True):
        st.session_state.run = True
```

Annexe D – Construction et traitement de la table de comparaison

```
header = ['instrument', 'optionPrice', 'strike', 'expiry', 'vol', 'type']

def read_data_createDataframe(res1, res2):
    try:
        dt1=res1['data'][0]['constituents']
        dt2=res2['data'][0]['constituents']
        tab1=[[i["optionRic"],i["optionPremium"],i["strike"],i["expiry"],i["volatility"],i["callPut"]] for i in dt1]
        tab2=[[i["optionRic"],i["optionPremium"],i["strike"],i["expiry"],i["volatility"],i["callPut"]] for i in dt2]

        dataframe1 = pd.DataFrame(tab1,columns=header)
        dataframe2 = pd.DataFrame(tab2,columns=header)
        dataframe1["expiry"] = pd.to_datetime(dataframe1["expiry"]).dt.strftime("%Y-%m-%d")
        dataframe2["expiry"] = pd.to_datetime(dataframe2["expiry"]).dt.strftime("%Y-%m-%d")
    except KeyError:
        dataframe2=None
        dataframe1=None
    return dataframe1, dataframe2

##### Merge the two dates on instrument

def merge_data(df1, df2, merge_column):
    df_merge=df1.merge(df2,on=merge_column,how='outer',indicator=True)
    return df_merge

##### Keep a single strike and expiry column

def clean_merged_data(df_merge):
    df_merge = df_merge.drop(columns=["strike_y", "expiry_y", "type_y"])
    df_merge = df_merge.rename(columns={"strike_x": "strike", "expiry_x": "expiry", "type_x": "type"})

    return df_merge

##### Split matched and unmatched instruments

def split_instruments(df_merge):
    df_final = df_merge[df_merge["_merge"] == "both"]
    df_nan = df_merge[df_merge["_merge"] != "both"]
    return df_final, df_nan

##### Compute relative volatility difference

def difference(vol1, vol2):
    return abs(vol1 - vol2) / vol2

##### Check if tolerance threshold is exceeded

def tolerance(diff, tolerance):
    return diff > tolerance

##### if tolerance is exceeded row = background-color: red

def highlight_tolerance(row, tolerance):
    styles = "background-color: red; color: black; font-weight: bold"

    if row["diff(%)"] > tolerance:
        return [styles] * len(row)
    else:
        return [""] * len(row)
```

Annexe E – Tableau de comparaison flouté des instruments dépassant un seuil de tolérance

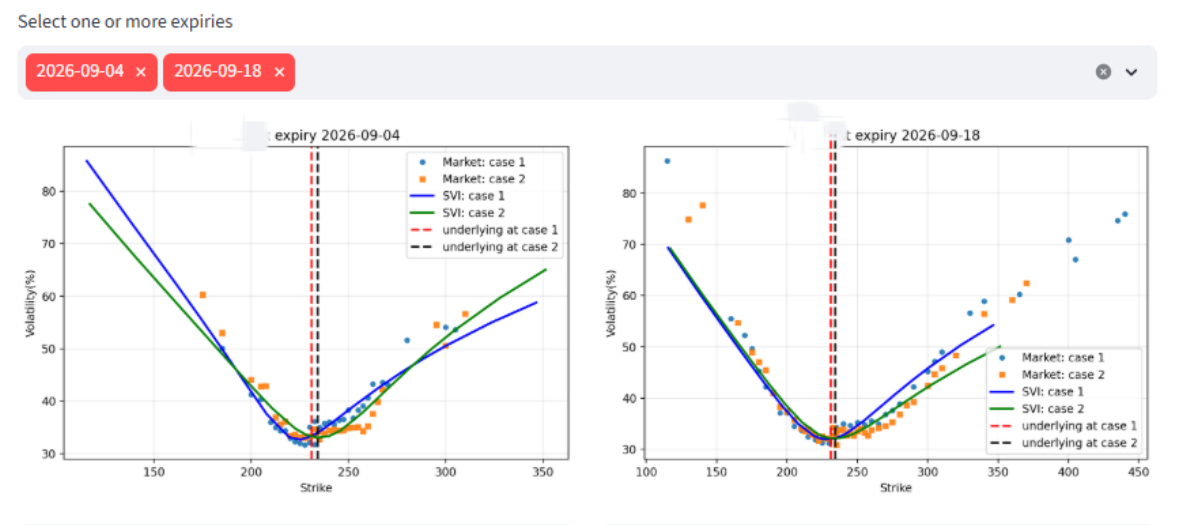
Constituent Table

 237 instruments out of 508 exceed the 2.0% tolerance threshold

	instrument	optionPrice_x	strike	expiry	vol_x	type	optionPrice_y	vol_y	_merge	diff(%)
0		6.250000	100.000000	2027-03-19	29.110723	PUT	6.725000	28.320320	both	2.790213
1		9.400000	100.000000	2027-06-18	31.474988	PUT	9.850000	31.856918	both	1.198892
2		14.150000	100.000000	2028-06-16	29.442066	PUT	14.650000	29.377115	both	0.221093
3		18.476000	100.000000	2029-06-15	28.300486	PUT	18.476000	29.452987	both	3.913017
4		None	None	None	None	None	0.012900	55.720407	right_only	None
5		0.755000	100.000000	2026-09-18	28.119608	PUT	1.040000	26.711435	both	5.271799
6		2.175000	100.000000	2026-10-16	27.344987	PUT	2.475000	31.871875	both	14.203454
7		22.939800	100.000000	2030-12-20	28.437340	PUT	22.939800	29.319890	both	1.009188
8		4.300000	100.000000	2026-12-18	28.464451	PUT	4.615000	30.061629	both	5.313011
9		12.225000	100.000000	2027-12-17	30.287434	PUT	12.650000	30.625287	both	1.103185

Annexe F – Comparaison graphique des volatilités de marché et du modèle SVI

Chart



Annexe G – Analyse de la répartition des contrats autour de l'ATM

nb points for case1 : 53 | nb points for case2 : 50

	Strikes <20% ATM	Around ATM 85.83-128.75	Strikes >20% ATM
nb points case1	20	28	5
nb points case2	20	25	5

nb points for case1 : 31 | nb points for case2 : 29

	Strikes <20% ATM	Around ATM 85.83-128.75	Strikes >20% ATM
nb points case1	0	28	3
nb points case2	0	26	3

Annexe H - Fonction de génération du graphique comparatif entre données de marché et courbes SVI

```
def create_plot(df_plot1, df_plot2, expiry, inst, und1, und2, df_svi1, df_svi2):

    df_plot1 = df_plot1.sort_values("strike")
    df_plot2 = df_plot2.sort_values("strike")

    fig, ax = plt.subplots(figsize=(8, 5))

    ##### Constituents
    ax.plot(df_plot1["strike"], df_plot1["vol"], marker="o", markersize=4, linestyle="None", label=f"Market: case 1", alpha=0.9)
    ax.plot(df_plot2["strike"], df_plot2["vol"], marker="s", markersize=4, linestyle="None", label=f"Market: case 2", alpha=0.9)
    ax.set_xlabel("Strike")
    ax.set_ylabel("Volatility(%)")

    expiry = expiry[0] if isinstance(expiry, list) else expiry

    ##### svi
    if expiry in df_svi1.index:
        svi_curve1 = df_svi1.loc[expiry]
        ax.plot(svi_curve1.index.astype(float), svi_curve1.values.astype(float), color="blue", linewidth=2, label=f"SVI: case 1")

    if expiry in df_svi2.index:
        svi_curve2 = df_svi2.loc[expiry]
        ax.plot(svi_curve2.index.astype(float), svi_curve2.values.astype(float), color="green", linewidth=2, label=f"SVI: case 2")

    ##### underlying
    ax.axvline(und1, linestyle="--", color="red", markersize=3, linewidth=2, label="underlying at case 1", alpha=0.9)
    ax.axvline(und2, linestyle="--", color="black", markersize=3, linewidth=2, label="underlying at case 2", alpha=0.9)

    ax.legend()
    ax.grid(True, alpha=0.3)
    ax.set_title((f'{inst} at expiry {expiry}'))
    return fig
```

Annexe I – Tableau des résultats SVI par échéance

SVI Table

SVI 1

	115.52	138.624	161.728	173.28	185.52
2026-08-28	100.637900	86.712933	73.016465	66.066969	58.066969
2026-09-04	85.727315	73.854630	62.141164	56.173363	50.173363
2026-09-11	71.538197	61.694441	52.036356	47.166930	42.166930
2026-09-18	69.295076	60.071761	51.203934	46.852609	42.852609
2026-09-25	66.605503	57.818709	49.428278	45.359006	42.359006
2026-10-02	63.551938	55.205897	47.281257	43.483370	39.483370
2026-10-16	56.649952	49.564413	43.108309	40.205119	37.205119
2026-11-20	58.237973	51.657554	46.191357	44.051497	42.051497
2026-12-18	52.604330	47.722025	43.894579	42.417126	41.417126
2027-01-15	49.421063	45.157437	41.771586	40.520852	39.520852
2027-01-22	48.066969	44.066969	40.666969	40.066969	39.066969

SVI 2

	117.095	140.514	163.933	175.6425	187.352
2026-08-28	96.762684	84.859521	73.312898	67.515796	61.515796
2026-09-04	77.522794	67.449775	57.709739	52.870052	48.870052
2026-09-11	59.806585	52.370394	45.432687	42.155416	39.155416
2026-09-18	69.168542	59.983811	51.174676	46.868569	42.868569
2026-09-25	68.227441	59.157507	50.447001	46.186840	42.186840
2026-10-02	66.092226	57.226088	48.669407	44.474206	40.474206
2026-10-16	60.947868	53.116663	45.779971	42.327809	39.327809
2026-11-20	55.677031	50.404389	46.095599	44.329960	42.329960
2026-12-18	53.126469	48.085606	44.057017	42.460414	41.460414
2027-01-15	51.455621	46.293111	42.304354	40.840593	39.840593
2027-01-22	50.066969	45.166969	41.333333	40.066969	39.066969

Annexe J – Analyse du Goodness of Fit

```
def classify_goodness_of_fit(values: list[float]) -> list:
    classifications = []

    for value in values:
        if value < 0.5:
            classifications.append("Good fit")
        elif value < 0.7:
            classifications.append("Moderate fit")
        else:
            classifications.append("Poor fit")

    return classifications
```

Annexe K – Résultats du Goodness of Fit par échéance

GodnessOfFit

The RMSE(Average Spread Explained) by Maturity measures the quality of the volatility surface fit for each individual expiry. Instead of evaluating an average error across the entire surface, a separate RMSE is computed for each maturity.

By expiry, the goodness-of-fit classification is as follows:

- < 0.5: Good fit
- 0.5 to 0.7: Moderate fit
- > 0.7: Poor fit

Goodness of Fit for Case 1

	Average Spread Explained	Min Strike	Max Strike	goodnessFit
2026-08-28	0.733918	190.000000	305.000000	Moderate fit
2026-09-04	0.814693	185.000000	305.000000	Moderate fit
2026-09-11	0.818570	190.000000	300.000000	Moderate fit
2026-09-18	0.772051	115.000000	440.000000	Moderate fit
2026-09-25	0.544831	165.000000	310.000000	Poor fit
2026-10-02	0.800506	185.000000	310.000000	Moderate fit
2026-10-16	0.746898	150.000000	400.000000	Moderate fit
2026-11-20	0.754883	115.000000	405.000000	Moderate fit
2026-12-18	0.795230	120.000000	460.000000	Moderate fit
2027-01-15	0.830363	115.000000	460.000000	Moderate fit

Goodness of Fit for Case 2

	Average Spread Explained	Min Strike	Max Strike	goodnessFit
2026-08-28	0.510919	195.000000	395.000000	Poor fit
2026-09-04	0.672917	175.000000	310.000000	Poor fit
2026-09-11	0.354330	205.000000	280.000000	Good fit
2026-09-18	0.716129	130.000000	370.000000	Moderate fit
2026-09-25	0.582670	165.000000	300.000000	Poor fit
2026-10-02	0.441648	190.000000	290.000000	Good fit
2026-10-16	0.700111	115.000000	380.000000	Moderate fit
2026-11-20	0.818547	115.000000	405.000000	Moderate fit
2026-12-18	0.806155	115.000000	460.000000	Moderate fit
2027-01-15	0.811578	105.000000	460.000000	Moderate fit

Annexe L – Informations de synthèse de la surface de volatilité

SurfaceInformation

```
▼ {  
  ▼ "constituentsInformation" : {  
    "initialCount" : 2146  
    "finalCount" : 498  
    "volatilityType" : "Implied"  
    "timestamp" : "Default"  
  }  
  ▼ "filtrationResults" : [  
    ]  
  }  
}
```

Annexe M – Exportation des résultats vers Excel

```
def export_to_excel(
    df_final: pd.DataFrame,
    df_nan: pd.DataFrame,
    df_tolerance: pd.DataFrame,
    df_svi_1: pd.DataFrame,
    df_svi_2: pd.DataFrame,
    df_goodness_1: pd.DataFrame,
    df_goodness_2: pd.DataFrame,
) -> bytes:
    """Export all result DataFrames to an Excel file."""
    output = BytesIO()

    with pd.ExcelWriter(output, engine="openpyxl") as writer:
        df_final.to_excel(
            writer,
            index=False,
            sheet_name="df_final",
        )

        df_nan.to_excel(
            writer,
            index=False,
            sheet_name="df_nan",
        )

        df_tolerance.to_excel(
            writer,
            index=False,
            sheet_name="df_tolerance",
        )

        df_svi_1.to_excel(
            writer,
            index=False,
            sheet_name="df_svi1",
        )

        df_svi_2.to_excel(
            writer,
            index=False,
            sheet_name="df_svi2",
        )

        df_goodness_1.to_excel(
            writer,
            index=False,
            sheet_name="df_goodnessfit1",
        )

        df_goodness_2.to_excel(
            writer,
            index=False,
            sheet_name="df_goodnessfit2",
        )

    return output.getvalue()
```

Annexe N – Rapport Excel consolidé contenant les résultats de comparaison et de calibration

Corporate													
	A	B	C	D	E	F	G	H	I	J	K	L	M
1	instrumen	option	strike	expiry	vol_x	type	optionPr	vol_y	_merge				
2	XXXXXXXX	4.63	100	2027-03-1	27.89947	PUT	4.91	28.05818	both				
3	XXXXXXXX	7.625	100	2027-06-1	31.58719	PUT	7.75	31.22359	both				
4	XXXXXXXX	12.4	100	2028-06-1	29.44761	PUT	12.575	29.31991	both				
5	XXXXXXXX	16.5559	100	2029-06-1	29.34236	PUT	16.8613	30.2464	both				
6	XXXXXXXX	0.0244	100	2026-08-2	34.21581	PUT	0.09	36.77111	both				
7	XXXXXXXX	0.39	100	2026-09-1	27.35941	PUT	0.495	27.28865	both				
8	XXXXXXXX	1.34	100	2026-10-1	29.92072	PUT	1.53	30.00075	both				
9	XXXXXXXX	21.5843	100	2030-12-2	29.91229	PUT	21.7151	30.45494	both				
10	XXXXXXXX	2.88	100	2026-12-1	28.99029	PUT	3.105	29.03409	both				
11	XXXXXXXX	10.425	100	2027-12-1	30.65271	PUT	10.475	30.20451	both				
12	XXXXXXXX	13.65	100	2028-12-1	27.8201	PUT	14.05	28.1176	both				
13	XXXXXXXX	17.675	100	2029-12-2	28.55898	PUT	17.825	28.46895	both				
14	XXXXXXXX	0.085	102	2026-08-2	36.44429	PUT	0.105	32.15086	both				
15	XXXXXXXX	0.57	102	2026-09-1	26.47852	PUT	0.705	26.35247	both				
16	XXXXXXXX	1.745	102	2026-10-1	29.57942	PUT	1.965	29.60947	both				
17	XXXXXXXX	0.1	104	2026-08-2	31.28232	PUT	0.14	28.12006	both				
18	XXXXXXXX	0.815	104	2026-09-1	25.51523	PUT	0.98	25.28057	both				
19	XXXXXXXX	0.135	106	2026-08-2	26.64512	PUT	0.26	25.9657	both				
20	XXXXXXXX	1.15	106	2026-09-1	24.53209	PUT	1.36	24.28197	both				
21	XXXXXXXX	2.88	106	2026-10-1	29.20563	PUT	3.135	28.95724	both				
22	XXXXXXXX	0.28	108	2026-08-2	24.41102	PUT	0.495	23.99872	both				
23	XXXXXXXX	1.64	108	2026-09-1	23.85201	PUT	1.89	23.46802	both				
24	XXXXXXXX	3.605	108	2026-10-1	29.0025	PUT	3.965	29.11445	both				
25	XXXXXXXX	8.575	110	2027-03-1	26.5375	PUT	8.825	26.23112	both				
26	XXXXXXXX	12	110	2027-06-1	30.40343	PUT	12.25	30.16817	both				
27	XXXXXXXX	0.61	110	2026-08-2	22.76555	PUT	0.96	22.53177	both				
28	XXXXXXXX	2.265	110	2026-09-1	23.01625	PUT	2.57	22.58729	both				
29	XXXXXXXX	4.53	110	2026-10-1	29.26994	PUT	4.915	29.27476	both				
30	XXXXXXXX	6.775	110	2026-12-1	28.74259	PUT	6.975	28.09684	both				
31	XXXXXXXX	15.15	110	2027-12-1	29.79102	PUT	15.3	29.43858	both				

Annexe O – Documentation en Markdown

```
## FR Français

### Présentation

Cette application Streamlit permet de comparer deux surfaces de volatilité calculées via l'API QPS (Quantitative Pricing Service).

L'outil récupère les données de surface de volatilité pour deux scénarios distincts (dates et Price Sides), compare les constituants retournés, évalue la qualité de calibration, affiche les smiles de volatilité et permet l'export de l'ensemble des résultats au format Excel.

### Fonctionnalités

- Import d'un modèle de requête JSON QPS
- Comparaison de deux surfaces de volatilité
- Affichage des prix spot des sous-jacents
- Analyse des écarts de volatilité
- Détection des instruments dépassant un seuil de tolérance
- Visualisation des points de marché et des courbes SVI
- Affichage des matrices de calibration SVI
- Analyse de la qualité de calibration (Goodness of Fit)
- Export complet des résultats vers Excel

### Utilisation de l'application

1. Importer le fichier JSON de requête, quel que soit son emplacement.
2. Saisir l'instrument à analyser.
3. Configurer :
   - La date du cas 1 et le Price Side du cas 1
   - La date du cas 2 et le Price Side du cas 2
4. Définir un seuil de tolérance.
5. Lancer l'exécution en cliquant sur le bouton **Run**.

> **Important :** Après avoir lancé l'application, si l'utilisateur modifie le seuil de tolérance, une date ou un Price Side, il devra cliquer de nouveau sur **Run** afin de recalculer et d'afficher les nouveaux résultats.
```

Ln 6, Col 1 Spaces: 3 UTF-8 CRLF { } Markd

Annexe P – Bibliothèques Python utilisées

```
#####librairies

import pandas as pd
import matplotlib.pyplot as plt
import os
from requests import request
import copy
import json
import requests
from datetime import datetime, date
from io import BytesIO
import streamlit as st
from pathlib import Path
import matplotlib.pyplot as plt
```

Annexe Q – Espace de travail au sein des locaux de LSEG à Paris

